

Course 1 : Divide and Conquer

Definition

A technique that solves a problem by :

- dividing the problem into smaller sub-problems;
- recursively solving each part;
- combining the obtained solutions.

General principle

A recursive algorithm usually contains :

- a base case;
- a reduction step;
- a combination step.

Key formulas

Recurrence relation :

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

where :

- a : number of sub-problems;
- b : reduction factor;
- $f(n)$: combination cost.

Example

Merge Sort :

$$T(n) = 2T(n/2) + O(n)$$

The array is divided into two halves.

The merge operation combines sorted sub-arrays.

Final complexity :

$$O(n \log n)$$



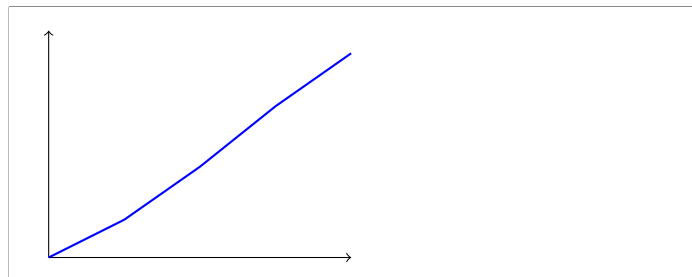
Course 2 : Complexity Analysis

Key idea

Algorithm complexity describes how the execution time grows according to the input size.

Common complexity classes

- $O(1)$: constant
- $O(\log n)$: logarithmic
- $O(n)$: linear
- $O(n \log n)$: quasi-linear
- $O(n^2)$: quadratic



Course 3 : Classical Algorithms

Example

Binary Search :

Sorted input :

[1, 3, 5, 8, 10, 15]

The search space is divided by two at each iteration.

Complexity :

$$O(\log n)$$

Warning

Binary search requires a sorted input array.

Important Exercises

Summary

- Compute algorithm complexity.
- Identify recurrence relations.
- Compare different approaches.
- Prove asymptotic bounds.

Weekly Summary

Key idea

Main points :

- Divide and conquer reduces complex problems.
- Recurrences describe recursive algorithms.
- Asymptotic analysis ignores constant factors.